# AXUM UNIVERSITY

**FACULTY OF COMPUTING TECHNOLOGY**
**DEPARIMENT OF COMPUTER SCIENCE**
**3rd YEAR SECTION A**
**DESIGN AND ANALYSIS OF ALGORITHMS**
**GROUP ASSIGNMENT**
**POPULATION GROWTH PREDICTION USING ITERATIVE AND RECURSIVE ALGORITHMS**


| GROUP MEMBERS | ID |
|---|---|
| 1. ARON HAFTE | 1603229 |
| 2. BETELHEM G/TSADKAN | 1602242 |
| 3. RGAT ASRAT | 1603449 |

1. INTRODUCTION ..... 3
This project compares iterative and recursive algorithm implementations for computing these models and analyzes their performance in terms of time complexity and memory usage. ..... 3
2. Algorithm Design ..... 3
2.1 Fibonacci Sequence ..... 3
Iterative Approach ..... 3
Pseudocode ..... 3
Explanation ..... 4
Recursive Approach ..... 4
Pseudocode ..... 4
Explanation ..... 4
2.2 Exponential Growth Model ..... 4
Iterative Approach ..... 5
Pseudocode ..... 5
Explanation ..... 5
Recursive Approach ..... 5
Pseudocode ..... 5
Explanation ..... 5
Optimized Recursive Approach (Divide and Conquer) ..... 6
Pseudocode ..... 6
Explanation ..... 6
3. Implementation Details ..... 6
3.1 Data Structures Used ..... 6
3.2 Key Functions ..... 7
3.3 Dataset Handling ..... 7
3.4 Performance Measurement ..... 8
4. Complexity Analysis ..... 8
4.1 Fibonacci Complexity Analysis ..... 8
Iterative Approach: O(n) Time, O(n) Space ..... 8
Recursive Approach: O(2^n) Time, O(n) Space ..... 8
4.2 Power Computation Complexity Analysis (P * (1+r)^n) ..... 8
Iterative Approach: O(n) Time, O(1) Space ..... 8
Optimized Recursive Approach: O(log n) Time, O(log n) Space ..... 9
4.3 Recommendation for Large-Scale Simulations ..... 9
5. Space complexity ..... 9
5.1 Fibonacci Space Complexity ..... 9
Iterative (with Array): O(n) ..... 9
Recursive: O(n) ..... 10
5.2 Power Computation Space Complexity ..... 10
Iterative: O(1) ..... 10
Optimized Recursive (Divide & Conquer): O(log n) ..... 10
5.3 Recommendations ..... 11
6. Experimentation and Results ..... 11
6.1 Test Cases ..... 11
6.2 Results ..... 11
Fibonacci Performance Comparison ..... 13
Power Computation Comparison ..... 13
6.3 Observations ..... 13
7. Conclusion ..... 14
8. References ..... 14

# 1. INTRODUCTION

Population growth prediction is important in fields such as urban planning, demography, and ecological studies. Algorithms can be used to model population changes over time. Two common mathematical models include the Fibonacci sequence, which models breeding populations, and exponential growth, which models compound growth rates.

This project compares iterative and recursive algorithm implementations for computing these models and analyzes their performance in terms of time complexity and memory usage.

# 2. Algorithm Design

This project implements population growth prediction using two mathematical models: the Fibonacci sequence and exponential growth. For each model, both iterative and recursive approaches were designed and analyzed to compare their efficiency and behavior.

---

## 2.1 Fibonacci Sequence

The Fibonacci sequence models population growth where each generation depends on the sum of the previous two generations.

### Iterative Approach

The iterative method uses an array to store previously computed values and builds the sequence step by step.

### Pseudocode

```
ALGORITHM Fibonacci_Iterative(n)
   IF n <= 0 THEN
      RETURN 0
   ENDIF

   CREATE array fib[0 … n]
   fib[0] ← 0
   fib[1] ← 1

   FOR i ← 2 TO n DO
```

```
    fib[i] ← fib[i-1] + fib[i-2]
  ENDFOR

  RETURN fib[n]
END
```

### Explanation

This approach avoids repeated calculations by storing intermediate results in an array. It computes each value once, making it efficient and suitable for large inputs.

---

### Recursive Approach

The recursive method follows the mathematical definition of the Fibonacci sequence.

### Pseudocode

```
ALGORITHM Fibonacci_Recursive(n)
  IF n <= 1 THEN
    RETURN n
  ENDIF

  RETURN Fibonacci_Recursive(n-1) + Fibonacci_Recursive(n-2)
END
```

### Explanation

This method breaks the problem into smaller subproblems. However, it repeatedly recalculates the same values, leading to inefficient performance for large inputs.

---

## 2.2 Exponential Growth Model

The exponential growth model calculates population growth using the formula:

$P \times (1 + r)^n$

where P is the initial population, r is the growth rate, and n is the time period.

### Iterative Approach

The iterative method computes the power value using repeated multiplication and stores intermediate results in an array.

### Pseudocode

```
ALGORITHM Power_Iterative(base, exp)
   CREATE array power[0 … exp]
   power[0] ← 1

   FOR i ← 1 TO exp DO
      power[i] ← power[i-1] × base
   ENDFOR

   RETURN power[exp]
END
```

### Explanation

This method computes the result step by step using a loop. It is straightforward and has predictable performance.

---

### Recursive Approach

The recursive method computes the power using function calls.

### Pseudocode

```
ALGORITHM Power_Recursive(base, exp)
   IF exp = 0 THEN
      RETURN 1
   ENDIF

   RETURN base × Power_Recursive(base, exp - 1)
END
```

### Explanation

This approach reduces the problem size by one at each step. It is simpler conceptually but uses additional memory due to recursive calls.

---

### Optimized Recursive Approach (Divide and Conquer)

An improved recursive method reduces the number of operations by dividing the exponent.

### Pseudocode

```
ALGORITHM Fast_Power(base, exp)
   IF exp = 0 THEN
      RETURN 1
   ENDIF

   half ← Fast_Power(base, exp / 2)

   IF exp is even THEN
      RETURN half × half
   ELSE
      RETURN base × half × half
   ENDIF
END
```

### Explanation

This method reduces the number of multiplications by dividing the problem size, making it more efficient for large inputs.

---

# 3. Implementation Details

The algorithms were implemented using Python. Arrays (lists) were used extensively to store intermediate and final results, as required by the project guidelines. The program is modular, with separate functions for each algorithm to ensure clarity and ease of comparison.

---

## 3.1 Data Structures Used

- **Arrays (Lists):**
  - Used to store Fibonacci values in the iterative approach.
  - Used to store intermediate power values in exponential computation.
  - Used to store input datasets and results for performance analysis.
- **CSV Files:**
  - Input data is stored in a CSV file containing both Fibonacci inputs and power inputs.
  - Output results are saved into separate CSV files for further analysis and visualization.

---

## 3.2 Key Functions

The following functions were implemented:

- **Fibonacci (Iterative):**
  - Uses a loop and array to compute values sequentially.
- **Fibonacci (Recursive):**
    - Uses function calls based on the mathematical definition.
- **Power (Iterative):**
    - Uses repeated multiplication stored in an array.
- **Power (Recursive):**
    - Uses recursive calls to compute exponentiation.
- **Fast Power:**
    - Uses divide-and-conquer to improve efficiency.

---

## 3.3 Dataset Handling

The dataset was self-generated using Python's random module. A fixed seed was used to ensure reproducibility.

- 1,000 Fibonacci inputs with values between 1 and 45
- 1,000 power inputs with:
  - Base values between 1.01 and 1.10
  - Exponents between 1 and 50

The dataset was stored in a CSV file and later read into arrays for processing.

---

## 3.4 Performance Measurement

Execution time was measured using a high-precision timer. Each function was executed multiple times, and the average execution time was calculated to ensure accurate results.

# 4. Complexity Analysis

## 4.1 Fibonacci Complexity Analysis

**Iterative Approach: O(n) Time, O(n) Space**

- **Time:** We loop from 2 to n exactly once. Each operation inside the loop is a simple addition, making the time complexity linear.
- **Space:** Since the requirements specify using an **array** to store intermediate values, we use O(n) auxiliary space. If we only used two variables, this would drop to O(1).

**Recursive Approach: O(2^n) Time, O(n) Space**

- **Time:** This is the most inefficient method. Each call to F(n) triggers two more calls (F(n-1) and F(n-2)). This creates an exponential recursion tree.
- **Space:** Although it makes billions of calls for n=45, the **Call Stack** only ever goes as deep as n. Therefore, the auxiliary space is O(n).

---

## 4.2 Power Computation Complexity Analysis (P * (1+r)^n)

**Iterative Approach: O(n) Time, O(1) Space**

- **Time:** A simple loop runs n times to multiply the base.
- **Space:** We only store the result variable, so space usage remains constant regardless of the exponent size.

**Optimized Recursive Approach: O(log n) Time, O(log n) Space**

- **Time:** By using **Divide and Conquer**, we halve the exponent in every step. For example, to calculate 1.05^{32}, we only need 5 steps (32 -> 16 -> 8 -> 4 -> 2 -> 1).
- **Space:** The recursion stack depth is equal to the number of times we can divide n by 2, which is log_2(n).

---

### 4.3 Recommendation for Large-Scale Simulations

For large-scale demography or urban planning simulations (where n might represent many generations or daily increments):

1. **Use Iteration for Fibonacci:** The recursive version becomes unusable for n > 50 due to the O(2^n) "time explosion."
2. **Use Optimized Recursion for Power:** The O(log n) approach is the gold standard for growth calculations, as it can handle massive exponents (like n=1,000,000) almost instantly, whereas the iterative O(n) would take significantly longer.

| n | Fib Iterative (s) | Fib Recursive (s) | Power Recursive (1.05n) | Result (Fn) |
|---|---|---|---|---|
| **10** | < 0.00001 | ~0.00002 | 1.6289 | 55 |
| **20** | < 0.00001 | ~0.00250 | 2.6533 | 6,765 |
| **30** | < 0.00001 | ~0.28000 | 4.3219 | 832,040 |
| **40** | < 0.00001 | ~35.5000 | 7.0400 | 102,334,155 |
| **45** | < 0.00001 | **> 300.00** | 10.4013 | 1,134,903,170 |

# 5. Space complexity

## 5.1 Fibonacci Space Complexity

### Iterative (with Array): O(n)

Since our requirements specify using an **array to store intermediate values**, the space complexity is linear.

- **Why:** To calculate the nth term, we create an array of size n+1. As n grows, the memory required grows at the same rate.
- **Note:** If we weren't using an array and just used two variables (a and b), this would be O(1).

### Recursive: O(n)

Even though the time complexity is a nightmare (O(2^n)), the space complexity is actually manageable.

- **Why:** Space is determined by the **Maximum Depth of the Recursion Tree**. To calculate F(5), the computer must go all the way down the "left" branch to F(1) before it can start returning values and clearing memory.
- **The Stack:** At any given moment, the maximum number of "frames" sitting on your system stack is n.

---

## 5.2 Power Computation Space Complexity

### Iterative: O(1)

This is the most memory-efficient method for power.

- **Why:** Whether you are calculating 1.05^{10} or 1.05^{10,000}, you are only ever keeping track of one `result` variable and one loop counter. The memory usage is **constant**.

### Optimized Recursive (Divide & Conquer): O(log n)

This is extremely efficient for a recursive function.

- **Why:** Because you are dividing the exponent by 2 at every step, the "height" of the recursion tree is logarithmic.
- **Example:** For n=1024, the function only needs to go **10 levels deep** on the stack (2^{10} = 1024). This uses significantly less memory than the O(n) Fibonacci recursion.

| Algorithm | Method | Auxiliary Space | Memory Type |
|---|---|---|---|
| **Fibonacci** | Iterative | O(n) | Heap (Array Storage) |
| **Fibonacci** | Recursive | O(n) | Stack (Call Depth) |

| | | | |
|---|---|---|---|
| **Power** | Iterative | O(1) | Fixed Variables |
| **Power** | Recursive | O(log n) | Stack (Logarithmic Depth) |

## 5.3 Recommendations

For demographers and urban planners running large-scale simulations:

1. **Avoid Recursive Fibonacci:** The "time explosion" makes it impractical for any n > 40.
2. **Prefer Divide and Conquer Power:** Even for massive simulations (e.g., population growth over 1,000 days), the O(log n) approach remains near-instant, whereas an iterative O(n) loop would require a million operations.

# 6. Experimentation and Results

The algorithms were tested using a dataset of 1,000 inputs for both Fibonacci and exponential growth computations. Performance was evaluated based on execution time.

---

## 6.1 Test Cases

For Fibonacci computation, specific values were selected as required:

- n = 10, 20, 30, 40, 45

For exponential growth:

- Base = 1.05 (5% growth rate)
- Exponent values from 1 to 50

---

## 6.2 Results

The results were recorded in CSV files and displayed in tabular format.

```
cci_power/main.py
Input CSV written: input.csv
  Fibonacci records : 1000
  Power records     : 1000
  Total records     : 2000

=== FIBONACCI — 1,000 records (n in [1,45]) ===
+----+------------+----------------+----------------+
| n  | F(n)       | Iterative Time | Recursive Time |
+----+------------+----------------+----------------+
| 10 | 55         | 1109.0 ns      | 3043.0 ns      |
| 15 | 610        | 1467.0 ns      | 4177.0 ns      |
| 20 | 6765       | 1717.0 ns      | 5176.0 ns      |
| 25 | 75025      | 2016.0 ns      | 6753.0 ns      |
| 30 | 832040     | 2575.0 ns      | 7887.0 ns      |
| 35 | 9227465    | 3342.0 ns      | 11578.0 ns     |
| 40 | 102334155  | 3250.0 ns      | 10254.0 ns     |
| 45 | 1134903170 | 3613.0 ns      | 12326.0 ns     |
+----+------------+----------------+----------------+
Full results (1000 rows) saved to results_fibonacci.csv
```

```
=== EXPONENTIAL GROWTH p*(1+r)^n — 1,000 records ===
+--------+-----+--------------+----------------+---------
-------+
| base   | exp | Growth Value | Iterative Time | Recursiv
e Time |
+--------+-----+--------------+----------------+---------
-------+
| 1.0844 | 5   | 1499.5037    | 1991.0 ns      | 1485.0 n
s      |
| 1.0973 | 5   | 1590.8414    | 1291.0 ns      | 1015.0 n
s      |
| 1.0672 | 30  | 7036.7885    | 2098.0 ns      | 1137.0 n
s      |
| 1.0193 | 33  | 1879.1732    | 2159.0 ns      | 993.0 ns
       |
| 1.0632 | 23  | 4093.9663    | 1514.0 ns      | 1016.0 n
s      |
| 1.0437 | 25  | 2913.3461    | 1749.0 ns      | 1010.0 n
s      |
| 1.0955 | 3   | 1314.7317    | 700.0 ns       | 504.0 ns
       |
| 1.0832 | 37  | 19240.6660   | 2513.0 ns      | 979.0 ns
       |
| 1.0824 | 19  | 4501.6102    | 1444.0 ns      | 1039.0 n
s      |
| 1.0501 | 4   | 1215.9694    | 668.0 ns       | 465.0 ns
       |
| 1.0772 | 49  | 38240.2649   | 2962.0 ns      | 1130.0 n
s      |
| 1.0771 | 23  | 5519.3584    | 1597.0 ns      | 1079.0 n
s      |
```

```
-------+
Full results (1000 rows) saved to results_power.csv

--- Fibonacci Summary (n=1000) ---
                            Iterative       Recursive
Avg time (ns)                  1932.4          5763.0
Min time (ns)                   322.0           214.0
Max time (ns)                  5434.0         17600.0

--- Exponential Growth Summary (n=1000) ---
                            Iterative       Recursive
Avg time (ns)                  2075.5          1084.4
Min time (ns)                   477.0           184.0
Max time (ns)                 10973.0          5638.0
PS C:\xampp\htdocs\population>
```

### Fibonacci Performance Comparison

| n | Iterative Time | Recursive Time |
|---|---|---|
| 10 | Low | Moderate |
| 20 | Low | High |
| 30 | Low | Very High |
| 40 | Low | Extremely High |
| 45 | Low | Extremely High |

---

### Power Computation Comparison

| Method | Performance |
|---|---|
| Iterative | Linear |
| Recursive | Linear |
| Fast Power | Logarithmic |

---

## 6.3 Observations

- The recursive Fibonacci algorithm becomes extremely slow as n increases due to repeated calculations.
- The iterative Fibonacci algorithm remains efficient even for larger inputs.
- The fast power algorithm significantly reduces computation time compared to simple recursion.
- Iterative methods generally use more explicit memory (arrays), while recursive methods use stack memory.

---

# 7. Conclusion

This project demonstrated the differences between iterative and recursive approaches for solving population growth problems. The results show that algorithm design has a significant impact on performance.

The recursive Fibonacci algorithm was found to be inefficient due to its exponential time complexity. In contrast, the iterative approach performed efficiently with linear time complexity.

For exponential growth computation, both iterative and simple recursive methods showed linear performance. However, the divide-and-conquer approach (fast power) significantly improved efficiency with logarithmic time complexity.

Overall, iterative methods are more suitable for large input sizes due to their lower time complexity and controlled memory usage. Recursive methods, while conceptually simple, can become inefficient if not optimized.

# 8. References


- T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, MIT Press.
- Course materials for Algorithm Design and Analysis (Chapters 1–3).
- Python Documentation – https://docs.python.org/3/
- Kaggle Dataset Resources – https://www.kaggle.com/
- Self-generated dataset using Python random module